Name of the module: crc8

Location in GIT repository: vspa_common\vspa-lib\crc8

Overview:

This module provides VCPU functions to 8-bit CRC generation of an input data sequence of length as specified by the corresponding size. The input data sequence is $\{m_0, m_1, \dots, m_{N-1}\}$, where m_0 is the least significant bit. The value of the CRC field shall be the 1s complement of

$$crc(D) = \big(M(D) \oplus I(D)\big) D^8 mod\ G(D)$$

Where $M(D) = m_0 D^{N-1} + m_1 D^{N-2} + \cdots + m_{N-2} D + m_{N-1}$

N is the number of bits over which the CRC is generated;
20 for 20 MHz, 21 for 40 MHz, and 23 for 80 MHz/160 MHz/80+80 MHz in VHT-SIG-A
34 in VHT-SIG-A

$I(D) = \sum_{i=N-8}^{N-1} D^i$ are initialized values that are added modulo 2 to the first 8 bits of the input data sequence

$G(D) = D^8 + D^2 + D + 1$ is the CRC generating polynomial

$crc(D) = c_0 D^7 + c_1 D^6 + \cdots + c_6 D + c_7$

The output crc sequence is $\{c_0, c_1, \dots, c_7\}$. The CRC field is transmitted with c_7 first.

Type of code:

- VCPU kernel function (1 function)

VCPU Function API:

```
extern uint_fast8_t   crc8_encode (
        uint32_t  data1,          // Least significant bit of data1 is m0
        uint32_t  data2,          // least significant bit of data2 is m32
                                  // data2 is only used if N = 34 bits
        uint_fast8_t  size,       // size of input data in bits
                                  // size = N = 20 , 21, 23, or 34
        );
```

Implementation plan:

This module provides VCPU functions to 8-bit CRC generation of an input data sequence of length as specified by the corresponding size. The implementation is performed in the following procedures:

1) Concatenate 8 bits of zeros to the input data sequence $\{m_0, m_1, \dots, m_{N-1}\}$ from the left side (most significant location). This means that the 8 zeros will start after the m_{N-1} bit
2) Flip the least significant 8 bits of the input data sequence $\{m_0, m_1, \dots, m_7\}$
3) Shift right the input data sequence until a bit which is set to 1 pops out. Note that the flipped m_0 bit will be the first bit to pop out
4) XOR the least significant 8 bits following the popped out 1 with 0xE0 (the generator polynomial)

5) Go back to step 3 until the input data sequence finished.

Below is the C implementation for the CRC function

```
uint_fast8_t crc8_encode(uint32_t data1, uint32_t data2, uint32_t size)
{
	uint_fast8_t crc;
	uint32_t gen_poly = 0xE0;
	uint32_t data_temp;
	int i;
	if (size == 34)
	{
		data_temp = data1 ^ 0x000000FF;
		for (i=0;i<2;i++)
		{
			if ((data_temp & 0x00000001) != 0)
			{
				data_temp >>= 1;
				if (((data2 >> i) & 0x00000001) != 0)
				{
					data_temp += 0x80000000;
				}
				data_temp ^= gen_poly;
			}
			else
			{
				data_temp >>= 1;
				if (((data2 >> i) & 0x00000001) != 0)
				{
					data_temp += 0x80000000;
				}
			}
		}
		for (i=0;i<32;i++)
		{
			if ((data_temp & 0x00000001) != 0)
			{
				data_temp >>= 1;
				data_temp ^= gen_poly;
			}
			else
			{
				data_temp >>= 1;
			}
		}
		crc = data_temp & 0xFF;
		crc ^= 0xFF;

	}
	else
	{
		data_temp = data1 ^ 0x000000FF;

		for (i=0;i<size;i++)
		{
			if ((data_temp & 0x00000001) != 0)
```

```
                {
                        data_temp >>= 1;
                        data_temp ^= gen_poly;
                }
                else
                {
                        data_temp >>= 1;
                }
        }
        crc = data_temp & 0xFF;
        crc ^= 0xFF;

    }

    return (crc);
}
```

DMEM requirements:

Function name	Variable	Minimum size (words)	Minimum alignment (words)	Allocated by
crc8_encode	data1	1	1	Caller
	data2	1	1	Caller
	crc_out	1	1	Caller
	size	1	1	Caller

Target efficiency:

TBD

Test plan:

- **Feature that are going to be tested:**
    - Input data sequence from specs document and compare the CRC output